RSpec provides the response.status value and there is, at least in theory, a have_http_matcher, which allows you to compare against logical values like :success and :error.

Let's talk about these three types of assertions in more detail.

You can use have_http_status to verify the HTTP response code sent back to the browser from Rails. Normally you'd use this assertion to ensure that your controller correctly distinguishes between success and redirect or error cases.

The value passed to have_http_status is usually one of four special symbols:

Symbol	HTTP Code Equivalent
:success	200–299
:redirect	300–399
:missing	404
:error	500–599

If you need to test for a more specific response, you can pass either the exact HTTP code number or one of the associated symbols defined by Rack.[3] Note that RSpec uses the codes defined by SYMBOL_TO_STATUS_CODE. The most common case I've had for specific HTTP codes is the need to distinguish between 301 permanent redirects (:moved-permanently) and other redirects.

When you expect the controller to redirect, you can use the redirect_to matcher to assert the exact nature of the redirect. The argument to redirect_to is pretty much anything Rails can convert to a URL, although the method's behavior is slightly different based on what the argument actually is. The code for redirect_to explicitly includes have_http_status(:redirect), so you don't need to duplicate that assertion.

If the argument to redirect_to is the name of a URL because it's a string or a method for a Rails named route, or because it's an object that has a Rails RESTful route, then the assertion passes if and only if the redirecting URL exactly matches the asserted URL. For testing purposes, Rails will assume that the application's local hostname is http://www.example.com. If that's too exact a test for your taste, you can pass a hash to redirect_to, which specifies the :controller, :action, and any parameters. If the argument is a hash, then assert_redirected_to checks only the exact key/value pairs in the hash; other parts of the URL are not checked.

3. https://github.com/rack/rack/blob/master/lib/rack/utils.rb